

Semáforo Inteligente de 3 Ruas

Pedro Ferreira, Rodrigo Soares e Virgínia Linhares

Junho 2025

Conteúdo

1	Introdução	3
2	Aplicações Possíveis	3
3	Circuito	4
4	Funcionalidades	5
4.1	Ciclo de 3 semáforos clássico	5
4.2	Sensor ultrassônico: distância e velocidade	6
4.3	Passadeira	8
4.4	Amarelo intermitente	9
5	Dificuldades encontradas	9
6	Sugestões de melhoria	10
7	Conclusão	10
8	Referências	10
9	Anexos	11
9.1	Configuração do sistema	11
9.2	Vídeos	11

1 Introdução

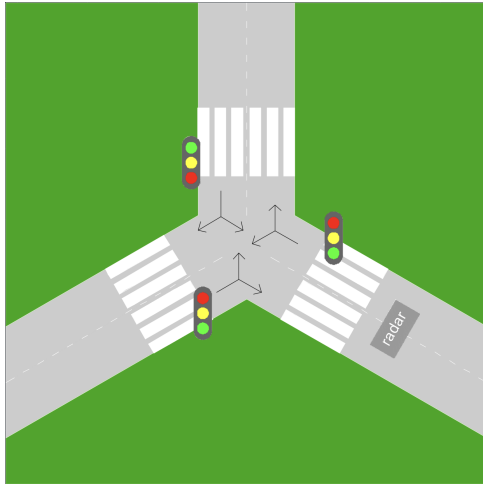


Figura 1: Esquema do cruzamento

O principal objetivo deste projeto foi desenvolver um sistema de semáforos avançado, mais eficiente e adaptável que os convencionais. Começamos por esquematizar o projeto com uma abordagem generalista, de forma a permitir uma fácil adaptação a cenários com um maior número de vias ou com diferentes regras de trânsito. Para isso, optou-se por um traçado em forma de 'Y' invertido, conforme ilustrado na Figura 1.

Cada uma das três ruas possui um semáforo equipado com um radar. A topologia permite que, a partir de qualquer uma das ruas, seja possível aceder às outras duas. Os semáforos são capazes de detetar tanto a velocidade dos veículos como a sua distância em relação ao cruzamento.

O sistema foi concebido para ser adaptável, permitindo configurar tanto o número de semáforos, como a duração dos ciclos e as condições que os podem interromper. Cada radar tem a capacidade de identificar veículos que excedam a velocidade permitida (um valor também configurável), acionando automaticamente o sinal vermelho nessa via e prolongando o tempo nas restantes ruas.

Adicionalmente, foram integradas passadeiras para peões em cada via, acompanhadas de botões para pedestres. Quando um peão pressiona o botão correspondente, o sistema atribui sinal vermelho à via no fim de um ciclo de semáforos, permitindo a travessia segura dos peões. Se não houver veículos próximos (isto é, se todos estiverem a uma distância superior a um valor configurável), o sistema antecipa a mudança do semáforo para vermelho, facilitando a travessia dos peões. Esta funcionalidade reforça a componente inteligente do sistema, promovendo maior segurança e fluidez tanto para condutores como para peões.

Por último, foi implementada uma funcionalidade de alerta, acionada através de um botão localizado numa central de controlo. Quando ativado, este modo força todos os semáforos a entrar em estado intermitente (piscar a amarelo), sinalizando atenção para todos os utilizadores da via.

2 Aplicações Possíveis

O sistema desenvolvido pode ser aplicado em diversos contextos urbanos, especialmente em locais onde é necessário reforçar a segurança rodoviária e otimizar o controlo do tráfego. As suas funcionalidades tornam-no particularmente útil em situações onde a deteção automática de veículos e a interação com peões são críticas.

As principais aplicações incluem:

- **Zonas escolares** – onde é fundamental garantir a travessia segura de crianças;
- **Áreas frequentadas por idosos** – permitindo tempos de travessia mais generosos e respostas mais rápidas a pedidos de passagem;
- **Locais com histórico de acidentes** – oferecendo controlo dinâmico e penalização automática de velocidades excessivas, contribuindo para a redução da sinistralidade;
- **Zonas com baixa visibilidade** – onde a deteção automática da aproximação de veículos pode compensar a falta de visibilidade, melhorando a segurança.

3 Circuito

Para simular o sistema, utilizámos LEDs, um sensor ultrassónico [1], um microcontrolador ESP32 [3] e um Arduino Uno [2].

Optámos por simular apenas um radar dado que o funcionamento seria idêntico para as restantes vias. No caso da passadeira, foi implementado um único botão que representa o acionamento de qualquer um dos botões disponíveis nas diversas vias, funcionando, na prática, como um *OR* lógico. Assim, independentemente de qual botão seja pressionado, o sistema regista o pedido de travessia e, no final do ciclo de semáforos em vigor, todos os semáforos passam a vermelho para permitir a passagem segura dos peões.

O ESP32 atua como unidade central de controlo do sistema: contém todo o código principal, bem como a ligação ao sensor. O Arduino foi utilizado unicamente para a comunicação com o LabVIEW, uma vez que a porta USB do ESP32 já se encontrava ocupada pelo IDE, usado para monitorizar a distância e a velocidade dos veículos.

O circuito implementado é relativamente simples. Os LEDs estão ligados aos pinos GPIO dos microcontroladores através de resistências ($R = 1\text{ k}\Omega$) que limitam a corrente (Figura 3b). Na Figura 2 é possível observar a existência de dois LEDs adicionais: um LED vermelho, conectado ao Arduino (Figura 3a), que indica quando o botão da passadeira é pressionado; e um LED amarelo, ligado ao ESP32, que permanece aceso quando os peões têm permissão para atravessar.

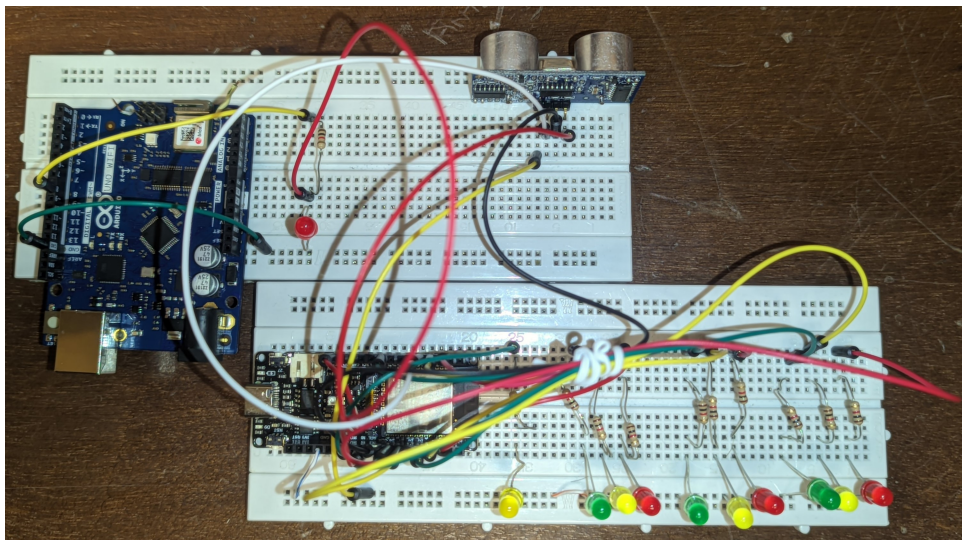
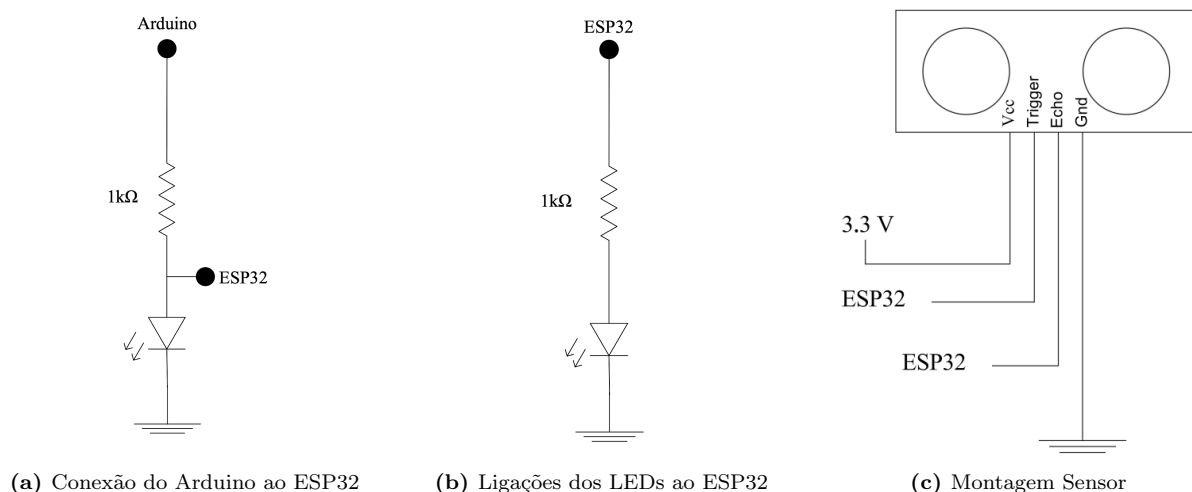


Figura 2: Montagem do circuito



4 Funcionalidades

A lógica de funcionamento do sistema foi implementada em MicroPython no ESP32, recorrendo ao modelo de programação assíncrona com a biblioteca `uasyncio`. No Arduino, utilizou-se a linguagem C para a comunicação com o LabVIEW. Nesta secção são descritas as principais funcionalidades do sistema, com excertos de código que ilustram o comportamento implementado.

Todas as funcionalidades descritas são executadas em tarefas assíncronas independentes, permitindo que diferentes partes do sistema operem em paralelo sem bloqueios.

A configuração do sistema, inicialização e encerramento encontra-se no Anexo, bem como os vídeos de funcionamento das diversas funções que serão apresentadas em seguida.

4.1 Ciclo de 3 semáforos clássico

O funcionamento base segue o ciclo tradicional de semáforos com três vias. Cada semáforo avança ciclicamente, garantindo que apenas uma via tem passagem autorizada de cada vez, enquanto as restantes se mantêm em vermelho. Este ciclo garante uma gestão simples e eficaz do tráfego num cruzamento em ‘Y’.

O código em seguida mostra a implementação desta lógica, dando uso à biblioteca `uasyncio`:

```
1 async def semaforo(verde, amarelo, vermelho):
2     #Semaforo Atual
3     await asyncio.sleep_ms(3000)
4     verde.value(False) ; amarelo.value(True)
5     await asyncio.sleep_ms(1000)
6     amarelo.value(False) ; vermelho.value(True)
7     await asyncio.sleep_ms(2000)
8
9 async def semaforos(todos_semaforos, tarefas, i = 0):
10     index = [0, 1, 2, 0]
11     while True:
12         verde, amarelo, vermelho = todos_semaforos[i]
13         verde_f, amarelo_f, vermelho_f = todos_semaforos[index[i+1]]
14         verde_t, amarelo_t, vermelho_t = todos_semaforos[i-1]
15         verde_f.value(False), amarelo_f.value(False), vermelho_f.value(True)
16         verde_t.value(False), amarelo_t.value(False), vermelho_t.value(True)
17         verde.value(True), amarelo.value(False), vermelho.value(False)
18         semaforo_i = asyncio.create_task(semaforo(verde, amarelo, vermelho))
19         tarefas[1] = semaforo_i
20         funcoes_i = asyncio.create_task(funcoes(verde, amarelo, vermelho, i,
21             semaforo_i))
22         tarefas[2] = funcoes_i
23         amarelo_intermitente_i = asyncio.create_task(amarelo_intermitente(
24             todos_semaforos, semaforo_i, funcoes_i))
25         tarefas[0] = amarelo_intermitente_i
26         i = (i + 1) % 3
27         while not amarelo_intermitente_i.done():
28             await asyncio.sleep_ms(0)
```

Listing 1: Código para controlo dos semáforos

Na função `semaforo`, define-se o comportamento de um semáforo individual: o sinal verde permanece ligado durante 3 segundos, seguido de 1 segundo em amarelo e 2 segundos em vermelho. Esta sequência simula o ciclo clássico de um semáforo.

A função `semaforos` é responsável por coordenar os três semáforos do cruzamento, garantindo que apenas um deles esteja ativo. O ciclo avança de forma circular entre as três vias.

Dentro desta função, são também iniciadas tarefas assíncronas associadas ao semáforo em funcionamento. Estas tarefas serão descritas em mais detalhe nas subsecções seguintes.

4.2 Sensor ultrassónico: distância e velocidade

Para implementarmos um sensor de distância e um radar, foi utilizado o sensor ultrassónico HC-SR04. Este sensor funciona emitindo um pulso de ultrassom através do pino TRIG e medindo o tempo que o eco demora a regressar ao pino ECHO após refletir num objeto (neste caso, num veículo).

O tempo medido (t) corresponde ao percurso de ida e volta da onda sonora. Assim, a distância ao objeto (d) é dada pela seguinte fórmula:

$$d = \frac{v \cdot t}{2}$$

Onde v é a velocidade do som no ar (aproximadamente 340 m/s à temperatura ambiente).

Para obter a velocidade de um veículo, são feitas duas medições de distância em instantes distintos.

$$v = \left| \frac{d_2 - d_1}{dt} \right|$$

```
1 async def funcoes(verde, amarelo, vermelho, i, semaforo_i):
2     global botao_passadeira, passadeira, d, v, dist, pos
3     while True:
4         if semaforo_i.done():
5             return
6         if v > 30 and not (amarelo.value() or vermelho.value()):
7             semaforo_i.cancel()
8             verde.value(False) ; amarelo.value(True) ; vermelho.value(False)
9             await asyncio.sleep_ms(500)
10            verde.value(False) ; amarelo.value(False) ; vermelho.value(True)
11            await asyncio.sleep_ms(6000)
12            while v > 30:
13                await asyncio.sleep_ms(500)
14            return
15        #passadeira
16        if d > 100 and not (amarelo.value() or vermelho.value()):
17            semaforo_i.cancel()
18            verde.value(False) ; amarelo.value(True) ; vermelho.value(False)
19            await asyncio.sleep_ms(1000)
20            verde.value(False) ; amarelo.value(False) ; vermelho.value(True)
21            passadeira = 0
22            LED_P.value(True)
23            while d > 100:
24                await asyncio.sleep_ms(500)
25            LED_P.value(False)
26            return
27        if passadeira and vermelho.value():
28            semaforo_i.cancel()
29            passadeira = 0
30            verde.value(False) ; amarelo.value(False) ; vermelho.value(True)
31            LED_P.value(True)
32            await asyncio.sleep_ms(7000)
33            LED_P.value(False)
34            return
35        await asyncio.sleep_ms(10)
36
37    def distancia(dt, c=1e-4):
38        #c - constante de conversao
39        return 340 * dt * c / 2
40
41    def velocidade(dt, dists):
42        #dists: array 1*2 que possui as distancias medidas com intervalo de tempo
43        dt
44        return abs((dists[1] - dists[0]) / dt)
```

```

45 def pulse_change(PulseIn):
46     global dt, pos
47     if PulseIn.value() > 0:
48         dt[pos] = ticks_us()
49     else:
50         dt[pos] = abs(dt[pos] - ticks_us())
51         pulse_event.set()
52
53 async def sensor(PulseIn):
54     global f, dt, dists, pos, v, D, d, V
55     D_N = len(D)
56     V_N = len(V)
57     D_n = 0
58     V_n = 0
59     await pulse_event.wait()
60     dists[pos] = dists[1] = distancia(dt[pos])
61     for d_i, d_e in enumerate(D):
62         D[d_i] = dists[pos]
63     v = velocidade(1/f, dists)
64     for v_i, v_e in enumerate(V):
65         V[v_i] = v
66     d = sum(D)/D_N
67     v = sum(V)/V_N
68     print("Distancia_ao_Sensor_(%d_medias):_%3d_cm_|_Velocidade_do_objeto_(%d_
        medias):_%3d_cm/s" % (D_N, d, V_N, v))
69     pos = (pos + 1) % 2
70     D_n = (D_n + 1) % D_N
71     V_n = (V_n + 1) % V_N
72     while True:
73         await pulse_event.wait()
74         dists[pos] = distancia(dt[pos])
75         D[D_n] = dists[pos]
76         V[V_n] = velocidade(1/f, dists)
77         d = sum(D)/D_N
78         v = sum(V)/V_N
79         print("Distancia_ao_Sensor_(%d_medias):_%3d_cm_|_Velocidade_do_objeto_(%d_
            medias):_%3d_cm/s" % (D_N, d, V_N, v))
80         pos = (pos + 1) % 2
81         D_n = (D_n + 1) % D_N
82         V_n = (V_n + 1) % V_N
83         await asyncio.sleep_ms(0)

```

Listing 2: Código Sensor

O fator de correção ($c = 1e-4$) serve para que o resultado final da função `distancia()` esteja em centímetros e consequentemente a velocidade esteja em centímetros por segundo.

A função `distancia` converte o tempo de ida e volta do pulso ultrassónico (`dt`) em distância, aplicando a fórmula apresentada anteriormente. A função `velocidade` calcula a velocidade média entre duas amostras de distância recolhidas com um intervalo temporal conhecido.

A função `pulse_change` regista o tempo de subida e descida do sinal ECHO para calcular a duração do pulso refletido. Este valor é posteriormente processado na função `sensor`, que calcula as distâncias e velocidades, atualizando valores médios para maior robustez.

A função `funcoes` avalia continuamente as medições em tempo real. Se um veículo ultrapassar o limite de velocidade definido ($v > 30$ cm/s), o semáforo da sua via muda imediatamente para vermelho, funcionando como mecanismo de controlo e penalização preventiva. Do mesmo modo, se não for detetado nenhum veículo dentro de uma distância mínima ($d > 100$ cm), o sistema assume que a via está livre, ativando a travessia de peões. Adicionalmente, esta função também trata o caso em que o botão de passadeira é pressionado, que iremos detalhar mais à frente.

4.4 Amarelo intermitente

O sistema dispõe de uma funcionalidade de alerta que pode ser ativada através de um botão físico ligado diretamente ao ESP32. Esta função destina-se a situações onde é necessário suspender temporariamente o funcionamento normal dos semáforos, como em operações de manutenção.

Ao ser acionado, o sistema cancela os ciclos de controlo dos semáforos e ativa todos os LEDs amarelos de forma intermitente. Este comportamento é comum em sistemas reais de semáforos, sinalizando um estado de atenção redobrada e enfatizando a necessidade de uma precaução por todos os utilizadores da via.

O funcionamento é controlado por uma tarefa assíncrona que verifica o estado do botão (neste caso embutido no ESP32). Quando detetada a ativação, os semáforos são desligados e os LEDs amarelos passam a piscar com um intervalo regular. Ao desativar o modo de emergência, pressionando novamente esse botão, os semáforos voltam ao seu estado normal.

```
1 async def amarelo_intermitente(todos_semaforos, semaforo_i, funcoes_i):
2     while botao_amarelo_intermitente.value():
3         if semaforo_i.done() and funcoes_i.done():
4             return
5         await asyncio.sleep_ms(10)
6     while not botao_amarelo_intermitente.value():
7         await asyncio.sleep_ms(10)
8     semaforo_i.cancel() ; funcoes_i.cancel()
9     todos_semaforos[0][0].value(False), todos_semaforos[1][0].value(False),
10     todos_semaforos[2][0].value(False)
11     todos_semaforos[0][1].value(True), todos_semaforos[1][1].value(True),
12     todos_semaforos[2][1].value(True)
13     todos_semaforos[0][2].value(False), todos_semaforos[1][2].value(False),
14     todos_semaforos[2][2].value(False)
15     last = ticks_ms()
16     while botao_amarelo_intermitente.value():
17         now = ticks_ms()
18         if now - last >= 1000:
19             last = now
20             todos_semaforos[0][1].value(not todos_semaforos[0][1].value())
21             todos_semaforos[1][1].value(not todos_semaforos[1][1].value())
22             todos_semaforos[2][1].value(not todos_semaforos[2][1].value())
23     while not botao_amarelo_intermitente.value():
24         await asyncio.sleep_ms(10)
```

5 Dificuldades encontradas

No decorrer da realização do projeto deparou-se com diversas dificuldades tais como:

1. indisponibilidade de placas Arduino nas primeiras aulas, que levou a usar o ESP32.
2. baixa compatibilidade do modelo do ESP32 (DFR0654) usado com os IDE's disponíveis.
3. a impossibilidade de armazenar o código na memória do ESP32 que, por sua vez, impedia a utilização da porta micro-usb para fazer comunicação com a interface LABVIEW. Nas últimas aulas, com recurso ao IDE uPyCraft, que foi desenvolvido pela própria fabricante do micro-controlador, conseguiu-se superar esta adversidade, ainda que o software fosse relativamente obsoleto.
4. a impossibilidade da utilização da porta micro-usb para efetuar comunicação com o computador via protocolo UART, uma vez que encontra-se reservada para a comunicação entre o micro controlador e o IDE. Desta forma, dado que não se possuía nenhum adaptador dos pinos RX e TX para USB, deparou-se na necessidade de usar uma placa Arduino, para efetuar o papel de intermediário na transmissão de informação entre o ESP32 e a interface em LABVIEW.

6 Sugestões de melhoria

Na sequência das dificuldades anteriormente mencionadas, não foi possível realizar todas as funcionalidades e características inicialmente planeadas, tais como:

1. a implementação de um radar em cada uma das três vias.
2. a criação de um modelo que permitisse fazer a simulação das três ruas, de uma forma de maquete inculindo os respetivos suportes para os diversos leds e radares.
3. criação de uma rotina que permitisse a emissão de som para auxiliar as pessoas invisuais na travessia da passadeira.
4. deteção da presença de uma ambulância, gerindo assim o ciclo dos semáforos de forma a dar prioridade à via em questão.
5. implementação de transmissão da velocidade que cada radar detetaria no computador, permitindo ainda fazer uma análise do historial de cada via para identificar qual ou quais apresentariam maior perigo.
6. redução do material utilizado, usando apenas uma ESP32 (possivelmente de outro modelo) ou um Arduino (que implicaria a adaptação do código).
7. implentação de um sistema que permitisse detetar a presença de peões, evitando assim a necessidade da utilização de botões para as passadeiras.

7 Conclusão

Foi desenvolvido um projeto para a gestão inteligente de três ruas, sendo que, apesar de não se ter cumprido com todos os objetivos inicialmente idealizados, este possui a habilidade de gerir de forma autónoma o fluxo das vias de uma forma que promove a segurança rodoviária, tal como desejado.

8 Referências

- [1] <https://www.sparkfun.com/ultrasonic-distance-sensor-hc-sr04.html>
- [2] <https://store.arduino.cc/products/arduino-uno-wifi-rev2>
- [3] <https://www.dfrobot.com/product-1590.html>

9 Anexos

9.1 Configuração do sistema

```
1 from time import ticks_ms, ticks_us, sleep
2 from machine import Pin, PWM
3 import uasyncio as asyncio
4
5 async def main(tarefas):
6     Sensor_PWM = asyncio.create_task(sensor(PulseIn))
7     tarefas[3] = Sensor_PWM
8     sistema = asyncio.create_task(semáforos(todos_semáforos, tarefas))
9     tarefas[4] = sistema
10    while True:
11        await asyncio.sleep_ms(1000)
12
13    verdes = [Pin(0, Pin.OUT), Pin(15, Pin.OUT), Pin(2, Pin.OUT)]
14    amarelos = [Pin(26, Pin.OUT), Pin(12, Pin.OUT), Pin(13, Pin.OUT)]
15    vermelhos = [Pin(25, Pin.OUT), Pin(4, Pin.OUT), Pin(14, Pin.OUT)]
16
17    todos_semáforos = [[verdes[0], amarelos[0], vermelhos[0]], [verdes[1], amarelos[1], vermelhos[1]], [
18        verdes[2], amarelos[2], vermelhos[2]]]
19
20    botao_amarelo_intermitente = Pin(27, Pin.IN, Pin.PULL_UP) #botao ativo a nível baixo
21    botao_passadeira = Pin(22, Pin.IN, Pin.PULL_UP)
22    passadeira = 0
23    LED_P = Pin(23, Pin.OUT)
24    LED_P.value(False)
25
26    botao_passadeira.irq(trigger=Pin.IRQ_RISING, handler=arduino)
27
28    PulseIn = Pin(36, Pin.IN, Pin.PULL_UP) #colocar respectivo pino de echo
29    h = 1e-5
30    f = 2
31    pwm = PWM(Pin(21), freq = f, duty_ns = int(f*h*1e9))
32    dt = [0.0, 0.0]
33    dists = [0.0, 0.0] #[d1,d2]
34    D = [0.0, 0.0, 0.0]
35    d = 0.0
36    V = [0.0, 0.0, 0.0]
37    pos = 0
38    v = 0.0
39
40    pulse_event = asyncio.ThreadSafeFlag()
41    PulseIn.irq(trigger=Pin.IRQ_RISING | Pin.IRQ_FALLING, handler=pulse_change)
42
43    tarefas = [0,0,0,0,0]
44    try:
45        asyncio.run(main(tarefas))
46    except KeyboardInterrupt:
47        print("A encerrar o Sistema...")
48    finally:
49        todos_semáforos[0][0].value(False), todos_semáforos[1][0].value(False), todos_semáforos[2][0].value(
50            False)
51        todos_semáforos[0][1].value(False), todos_semáforos[1][1].value(False), todos_semáforos[2][1].value(
52            False)
53        todos_semáforos[0][2].value(False), todos_semáforos[1][2].value(False), todos_semáforos[2][2].value(
54            False)
55        asyncio.core.cur_task = None
56        pwm.deinit()
57        PulseIn.irq(handler=None)
58        botao_passadeira.irq(handler=None)
59        print("Processos terminados:", end = "")
60        for tarefa in tarefas:
61            tarefa.cancel()
62            sleep(0.5)
63            print("|", end = "")
64        print("\n")
65        sleep(1)
```

9.2 Vídeos

[Link para a pasta no OneDrive](#)